Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

RAPORT

la Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: „Funcții, metode și importuri pe web”

A efectuat: st. gr. SI-262 Bodaci Bogdan
A verificat: asistent universitar Dumitrașcu Marius

Chișinău – 2026

1. SARCINA LUCRĂRII

Scopul prezentei lucrări de laborator este exersarea și aprofundarea a trei concepte fundamentale din limbajul Python — importurile de module, apelarea metodelor/atributelor obiectelor și scrierea funcțiilor proprii — aplicate în contextul comunicării programatice cu resurse web reale.

Obiective specifice de învățare și dezvoltare:

- **Importul de module:** Utilizarea diferitelor stiluri de import (biblioteca standard vs. pachete externe instalate via pip), crearea de alias-uri și tratarea excepțiilor de tip ImportError.
- **Metode și atribute ale obiectelor HTTP:** Inspectarea răspunsurilor obținute prin cereri GET/HEAD/POST, analizarea codurilor de stare, antetelor HTTP, cookie-urilor și gestionarea sesiunilor cu User-Agent personalizat.
- **Dezvoltarea funcțiilor proprii:** Organizarea codului modular în funcții cu parametri, valori implicite, adnotări de tip (type hints), docstrings și tratarea robustă a erorilor de rețea fără întreruperea execuției.
- **Integrarea modulelor standard Python:** Utilizarea modulelor urllib.parse, re, html.parser, hashlib, json, socket și ssl pentru parsare HTML, analiză DNS, verficarea certificatelor SSL și hash-uri de date.
- **Construirea unui modul propriu și proiectul final:** Structurarea codului într-un modul reutilizabil (webtools.py), apelarea lui dintr-un script principal (main.py) prin linia de comandă (argparse) și generarea rapoartelor finale în format CSV și JSON.

> **Site-uri țintă și Reguli de Securitate/Conformitate:**
>
> • Site-uri utilizate: https://cybercor.org (interogări de citire GET/HEAD) și https://httpbin.org (testare cereri cu date/antete).
>
> • Regula Timeout: Fiecare cerere HTTP folosește obligatoriu parametrul timeout=10 pentru prevenirea blocării programului.
>
> • Regula Pause: Între cereri succesive repetate se introduce o pauză de timp de cel puțin 1 secundă (time.sleep(1)).
>
> • Bune practici: Respectarea cu strictețe a scopului read-only pentru site-ul principal și evitarea scanărilor neautorizate.

Structura detaliată a exercițiilor pe părți:

Partea	Descriere Exerciții și Subiecte Redate	Fișier Rezultat
Partea 1 (Ex 1–8)	Importuri (requests vs urllib, alias-uri, inspectare dir/help, cronometrare perf_counter, tratare ImportError).	part1_imports.py
Partea 2 (Ex 9–20)	Metode & Atribute HTTP (status_code, headers, cookie-uri, sesiuni cu User-Agent, redirecționări, feliere HTML).	part2_methods.py
Partea 3 (Ex 21–	Funcții Proprii (type hints, docstrings, argumente	part3_functions.py

34)	implicite/*args/**kwargs, verficare antete securitate, robots.txt).	
Partea 4 (Ex 35–43)	Module Standard (urlparse/urljoin, re.findall, clasa HTMLParser, sha256, socket DNS, cert_days_left SSL).	part4_modules.py
Partea 5 (Ex 44–50)	Modul Propriu & Raport Final (creare webtools.py, main.py cu argparse, bloc __main__, generare report.csv și report.json).	webtools.py / main.py

Partea 1 “imports”:

Deci pentru inceput am creat inceputul folosind regula care mi-a fost pusa:

```
Session  Actions  Edit  View  Help
  GNU nano 9.0                          part1_imports.p
# Laborator : functii, metode  si importuri pe web
# Student : Bodaci Bogdan

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10 #SECUNDE
```

Exercitiul 1. Primul import. Scrieți `import requests` și afișați `requests.__version__`. Apoi scrieți `import urllib.request`. Într-un comentariu, explicați de ce `requests` a avut nevoie de `pip install`, iar `urllib` nu.

Dupa am facut exercitiul 1 care trebuia doar sa gasesc versiunea python, cu ajutorul la import request si urlllib.

In python asta arata asa:

```
#Exercitiul 1
import requests
import  urllib.request
print("Veriunea afisata: ", requests.__version__)

#urllib este o biblioteca standart inclusa in python
#request este o biblioteca third-party dezvoltata extern de o comunitate
#de aia requestul are nevoie de pip install
```

Ea da outputul asta:

```
→ python part1_imports.py
Veriunea afisata:  2.32.5
```

Exercitiul 2. Două stiluri de import. Descărcati pagina `BASE_URL` cu `requests.get()` si afisati codul de stare. Apoi rescrieti acelasi script folosind `from requests import get`. Într-un comentariu, scrieti câte un avantaj al fiecărui stil.

Dupa care vine exercitiul 2, in el a trebuit sa folosesc import requests la fel si dupa sa folosesc requests.get() pentru a ajunge la status_code la situl de baza care a fost cybercor.org, prima data sa folosesc requests.get() si a doua oara form requests import get si sa scriu statusul la site.

Aici este exercitiul in python realizat:

```python
#Exercitiul 2

import requests
import urllib.request
response = requests.get(BASE_URL)
print("Stilul 1 - status code: ", response.status_code)
from requests import get
response = get(BASE_URL)
print("Stilul 2 -status code: ", response.status_code)
```

Sunt si comentarii in cod pe care le-am facut dar sunt prea mari pentru screenshot asa ca nu am putut sa le afisez.

Aici e outputul:

```
Stilul 1 - status code:  200
Stilul 2 -status code:  200
```

Exercitiul 3. Alias-uri. Importati requests cu un alias: `import requests as rq`. Descărcati pagina principală cu `rq.get()`. Într-un comentariu, spuneti când un alias face codul mai usor de citit si când îl face mai greu de citit.

In exercitiul 3 a trebuit sa importez request ca rq, si de fiecare data cand doresc sa folosesc requests, in locul lui folosesc rq.

Aici e codul in python:

```python
#Exercitiul 3

import requests as rq
response = rq.get(BASE_URL)
print(response)

#Alias-ul rq e util cand numele librariei este lung si vrei sa scrii codul mai rapid
#Poate fi mai greu de citit daca cineva strain se uia pe cod si nu stie ce inseamna
```

Si aici e outputul:

```
<Response [200]>
```

Exercitiul 4. Doar biblioteca standard. Descărcați pagina principală folosind doar `urllib.request.urlopen()`. Afișați atributul `.status` al răspunsului și primele 200 de caractere din corp.

> **Indiciu:** `.read()` returnează octeți (bytes). Folosiți `.decode("utf-8")` ca să îi transformați într-un șir de caractere.

In exercitiul 4 a trebuit sa folosesc doar biblioteca standard unde am important urllib.requests si din aceasta biblioteca am folosit requests, am fost nevoit sa folosesc si headers daca nu imi dadea eroare, si am printat doar cele 200 de caractere conform conditiei.

Aici e codul tot cu indiciu si cu tot:

```python
#Exercitiul 4
print()
import urllib.request
req = urllib.request.Request(BASE_URL, headers={'User-Agent' : 'Mozilla/5.0'})
response = urllib.request.urlopen(req)
print(response.status)
bytes = response.read()
text = bytes.decode("utf-8")
print(text[:200])
```

Aici e outputul:

Prima 200 e status.

```
200
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <!-- TODO: Set the document title to the na
```

Exercitiul 5. Priviți în interiorul unui modul. Rulați `print(dir(requests))`. Alegeți trei nume din rezultat și scrieți, într-un comentariu, dacă fiecare este o funcție, o clasă sau un modul.

In exercitiul 5 a trebuit sa ma uit in interiorul unui modul cu ajutorul la dir care mi-a aratat totul ce e inauntrul la requests, dupa care am ales 3 simple pe care pot sa le explic.

Aici e codul in python:

```python
#Exercitiul 5
print()
import requests
print(dir(requests))

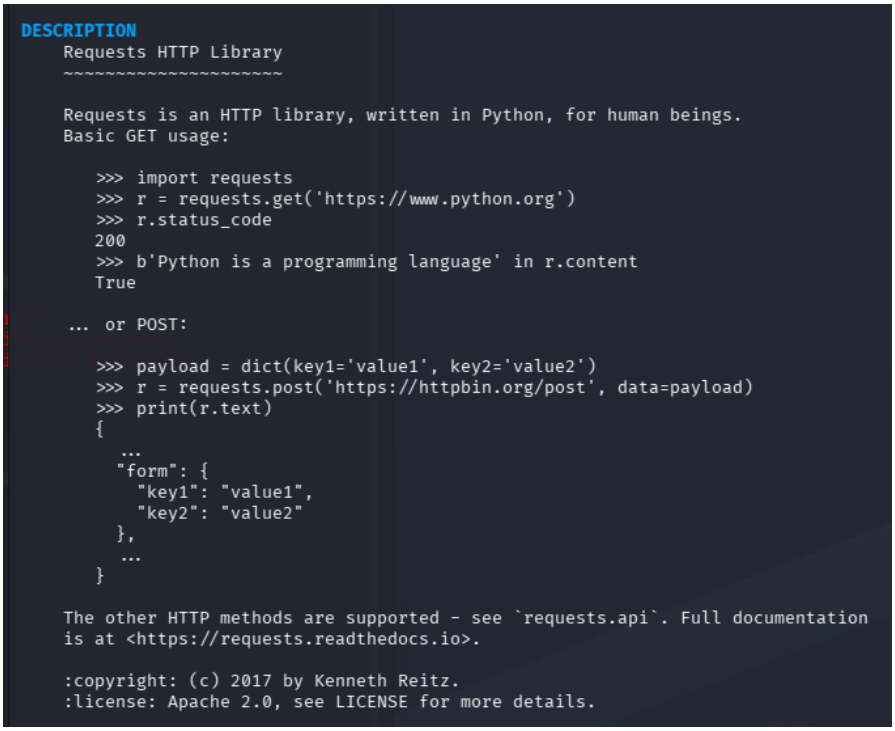
#Din lista obtonute am ales 3 nume:
#1.(get) care este o functie(o apelam sa facem o cerere)
#2.(Response) care este o clasa(obiectul care il primim ca raspuns)
#3.(exceptions) care este un modul intern(contine tipurile de erori)
```

Aici e outputul:

```
['ConnectTimeout', 'ConnectionError', 'DependencyWarning', 'FileModeWarning', 'HTTPEr
ror', 'JSONDecodeError', 'NullHandler', 'PreparedRequest', 'ReadTimeout', 'Request',
'RequestException', 'RequestsDependencyWarning', 'Response', 'Session', 'Timeout', 'T
ooManyRedirects', 'URLRequired', '__author__', '__author_email__', '__build__', '__bu
iltins__', '__cached__', '__cake__', '__copyright__', '__description__', '__doc__', '
__file__', '__license__', '__loader__', '__name__', '__package__', '__path__', '__spe
c__', '__title__', '__url__', '__version__', '_check_cryptography', '_internal_utils'
, 'adapters', 'api', 'auth', 'certs', 'chardet_version', 'charset_normalizer_version'
, 'check_compatibility', 'codes', 'compat', 'cookies', 'delete', 'exceptions', 'get',
 'head', 'hooks', 'logging', 'models', 'options', 'packages', 'patch', 'post', 'put',
 'request', 'session', 'sessions', 'ssl', 'status_codes', 'structures', 'urllib3', 'u
tils', 'warnings']
```

Exercitiul 6. Cititi documentatia. Rulati `help(requests.get)`. Găsiti parametrul care setează timpul maxim de asteptare și folositi-l într-o cerere.

In exercitiul 6 a trebuit sa rulez help(requests.get) si sa gasesc comanda care seteaza timpul maxim de asteptare pentru ca sa il folosesc intr-o cerere:



```
DESCRIPTION
    Requests HTTP Library
    ~~~~~~~~~~~~~~~~~~~~~

    Requests is an HTTP library, written in Python, for human beings.
    Basic GET usage:

       >>> import requests
       >>> r = requests.get('https://www.python.org')
       >>> r.status_code
       200
       >>> b'Python is a programming language' in r.content
       True

    ... or POST:

       >>> payload = dict(key1='value1', key2='value2')
       >>> r = requests.post('https://httpbin.org/post', data=payload)
       >>> print(r.text)
       {
         ...
         "form": {
           "key1": "value1",
           "key2": "value2"
         },
         ...
       }

    The other HTTP methods are supported - see `requests.api`. Full documentation
    is at <https://requests.readthedocs.io>.

    :copyright: (c) 2017 by Kenneth Reitz.
    :license: Apache 2.0, see LICENSE for more details.
```

Mai departe am facut aceasta programa in python:



```
#Exercitiul 6
print()
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
print("status code este:", response.status_code)

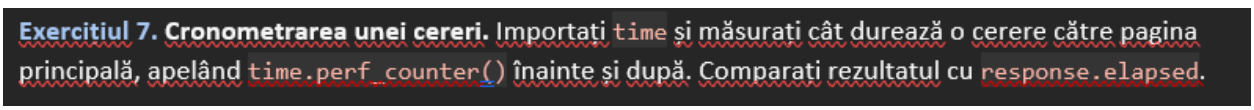
#Parametrul care seteaza timpul maxim este TIMEOUT
```

Si aici este outputul:



```
status code este: 200
```

Urmatorul:



Exercitiul 7. Cronometrarea unei cereri. Importați `time` și măsurați cât durează o cerere către pagina principală, apelând `time.perf_counter()` înainte și după. Comparați rezultatul cu `response.elapsed`.

Aici a trebuit sad au import time pentur a masura cat timp ia ca sa ruleze programul.

Aici este programul in python:

```python
#Exercitiul 7

import time
import requests
t_start = time.perf_counter()
response = requests.get(BASE_URL, timeout=TIMEOUT)
t_end =  time.perf_counter()
durata_noastra = t_end - t_start
durata_server = response.elapsed.total_seconds()
print(f"Timp masurat de catre noi : {durata_noastra:.4f} secunde")
print(f"Timp inregistrat de server : {durata_server:.4f} secunde")

#perf_counter masoarea tot timpul procesului
#response.elapsed masoara doar timpul din momentul trimiterii pana la primirea primi
```

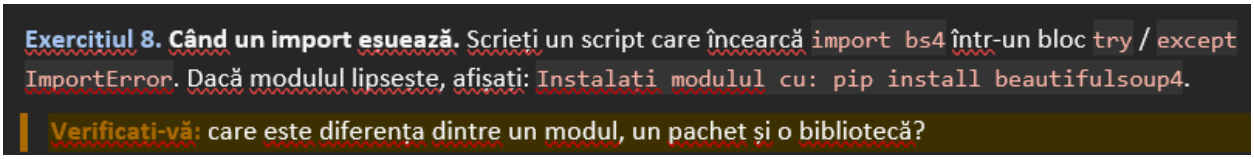
Aici este outputul:

```
Timp masurat de catre noi : 0.1279 secunde
```

Care la fel si a masurat timpul de inregistrare pe server:

```
Timp inregistrat de server : 0.1258 secunde
```

Si ultimul este exercitiul 8:



In exercitiul 8 a trebuit sa dau import bs4 intr-un bloc try/except si la except sa ii dau ImportError, daca modulul lipseste sa se afiseze “Instalati modulul cu pip: pip install beautifulsoup4”, care nu mi-a dat eroare.

Aici e codul in python:

```python
#Exercitiul 8

try:
    import bs4
except ImportError:
    print("Instalati modul cu: pip install beautifulsoup4")
```

Si ea nu da output, din vina ca acest import bs4 il am instalat deja cred.

Partea 2 “methods”:

Exercitiul 9. Elementele de bază ale răspunsului. Descărcați pagina principală și afișați `response.status_code`, `response.ok`, `response.url` și `response.encoding`. Lângă fiecare linie, scrieți un comentariu care spune dacă este un atribut sau o metodă.

In exercitiul 9 a trebuit sa descarc pagina principala si sa o afisez cu ajutorul la comenzile aratate mai sus si langa fiecare linie trebuia sa scriu un comentariu.

Aici e codul:

```python
#Exercitiul 9

import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
print("status_code:", response.status_code) #Atribut, deoarece este o valoare sau nu
print("ok:", response.ok) #Atribut, deoarece este True sau False
print("url:", response.url) #Atribut, deoarece este un text cu adresa
print("encoding:", response.encoding) #Atribut, deoarece este tipul de codificare
```

Aici e outputul:

```
status_code: 200
ok: True
url: https://cybercor.org/
encoding: utf-8
```

Exercitiul 10. Tratarea erorilor cu o metodă. Apelați `response.raise_for_status()` pe pagina principală. Apoi trimiteți o cerere către `BASE_URL + "/this-page-does-not-exist"` și prindeți excepția `requests.HTTPError`. Afișați un mesaj prietenos în loc să lăsați programul să se oprească cu eroare.

In exercitiul 10 a trebuit sa fac un try si except din nou, unde cand da ajunge la except sa dea requests.HTTPError, daca nu ajunge la BASE_URL + “/this-page-does-not-exist”.

Aici e codul:

```python
#Exercitiul 10

import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
response.raise_for_status()
try:
   bad_res = requests.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT)
   bad_res.raise_for_status()
except requests.HTTPError as e:
    print("Pagina nu exista! Am prins eroarea fara sa crape programul:", e)
```

Mie nu mi-a dat nici o eroare de aia ca am ajuns cu success.

Exercitiul 11. Toate antetele. Parcurgeti `response.headers.items()` și afișați fiecare antet în forma `Nume: valoare`.

In exercitiul 11 a trebuit sa arat toate antenele cu ajutorul la response.headers.items().

Aici e codul:

```python
#Exercitiul 11

import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
print("Toate antenele:")
for cheie, valoare in response.headers.items():
   print(f"{cheie}: {valoare}")
```

Am folosit un f-string ca sa fie mai usor.

Aici e outputul:

```
Toate antenele:
Date: Tue, 10 Nov 2026 19:27:34 GMT
Content-Type: text/html; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Cache-Control: no-cache
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.clou
dflare.com/report/v4?s=oqLW46fvHaVA5KEtUokjfXbcjX89nOL4bzRxTMWrWA0ulsQQPGv8uWd%2B20b6
Hbn7HG%2FGxSEXY4BN8S7X1nockOE%2FD73Do5NYPW%2F3WZCEiZGdKbAE%2F3Zi5nBLNfJ6V7Y%3D"}]}
Last-Modified: Wed, 14 Oct 2026 19:03:10 GMT
X-Powered-By: Express
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
cf-cache-status: DYNAMIC
Content-Encoding: br
Server: cloudflare
CF-RAY: a587748d6e0ae429-OTP
alt-svc: h3=":443"; ma=86400
```

Exercitiul 12. Metode de dicționar. Folositi `response.headers.get()` pentru a afișa antetele `Server` și `Content-Type`, cu `"lipseste"` ca valoare implicită. Apoi încercați `response.headers.get("content-type")`, cu litere mici. Ce observați?

In exercitiul 12 trebuie sa folosesc response.headers.get pentru a afista antenele Server si Content-Type cu “lipseste” ca valoare implicita, dupa trebuie sad au fix aceasi comanda cu content-type cu litere mici.

Aici e codul:

```
#Exercitiul 12
print("")
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
server_hdr = response.headers.get("Server","lipseeste")
content_type_hdr = response.headers.get("Content-Type", "lipseste")
lower_hdr = response.headers.get("content-type", "lipseste")
print(f"Server: {server_hdr}")
print(f"Content-Type: {content_type_hdr}")
print(f"content-type: {lower_hdr}")
```

Am scris un comentariu inauntru cu ce am observant inauntru la cod.

Aici e outputul:

```
Server: cloudflare
Content-Type: text/html; charset=utf-8
content-type: text/html; charset=utf-8
```

Exercitiul 13. Înlănțuirea metodelor de șir. Numărați de câte ori apare cuvântul `cyber` pe pagina principală, cu `response.text.lower().count("cyber")`. Într-un comentariu, explicați de ce puteți înlănțui `.lower()` și `.count()`.

In exercitiul 13 trebuie sa vad de cate ori cuvantul cyber apare pe site cu ajutorul la comanda response.text.lower(“cyber”) si dupa sa arat de cate ori a fost aratat. Am folsoit f-string.

Aici e codul in python:

```
#Exercitiul 13
print("")
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
numar_cyber = response.text.lower().count("cyber")
print(f"Cuvantul 'cyber' apare de {numar_cyber} ori pe pagina.")
```

Am lasat si un comentariu in cod in care am explicat cum merge totul.

Aici e outputul: `Cuvantul 'cyber' apare de 9 ori pe pagina.`

Exercitiul 14. Găsirea titlului. Folosiți `.find()` și felierea (slicing) pe `response.text` pentru a extrage textul dintre `<title>` și `</title>`. Curățați-l cu `.strip()` și afișați-l.

Indiciu: `start = html.find("<title>") + len("<title>")`

In exercitiul 14 a trebuit sa folosesc .find() pentru a extrage titlul paginii dintre <title> si </title>.

Aici e codul:

```
#Exercitiul 14
print("")
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
html = response.text
start = html.find("<title>") + len("<title>")
end = html.find("</title>")
if start ≠ -1 and end ≠ -1:
   titlu = html[start:end].strip()
   print("Titlul extras din pagina", titlu)
```

Aici e outputul:

```
Titlul extras din pagina CYBERCOR
```

Exercitiul 15. Numărarea liniilor. Împărțiți codul HTML în linii cu `.splitlines()`. Afișați câte linii sunt și lungimea celei mai lungi linii, folosind `max(lines, key=len)`.

In exercitiul 15 am trebuit sa numar cate linii cu ajutorul la .splitlines() in care una numara numarul de linii iar a doua numara linia cea mai lunga linie si cate caractere are.

Aici e codul:

```
#Exercitiul 15
print("")
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
html = response.text
linii = html.splitlines()
cea_mai_lunga = max(linii, key=len) if linii else ""
print(f"Numar total de linii: {len(linii)}")
print(f"Lunfimea celei mai lungi linii: {len(cea_mai_lunga)} caractere")
```

Aici e outputul:

```
Numar total de linii: 32
Lunfimea celei mai lungi linii: 218 caractere
```

Exercitiul 16. Verificarea HTTPS. Folositi `.startswith()` pentru a verifica dacă `response.url` începe cu `https://`. Afișați `Conexiune securizată` sau `Conexiune nesecurizată`.

In exercitiul 16 a trebuit sa verific conexiunea la BASE_URL cu ajutorul .startswith() daca e https://, daca nu e asa e nesecurizata.

Aici e codul:

```
#Exercitiul 16
print("")
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
if response.url.startswith("https://"):
   print("Conexiune securizata")
else:
    print("Conexiune nesecurizata")
```

Aici e outputul:

```
Conexiune securizata
```

Exercitiul 17. Urmărirea redirecționărilor. Trimiteți o cerere către `http://cybercor.org` (HTTP simplu, fără S). Parcurgeți `response.history` și afișați codul de stare și URL-ul fiecărei redirecționări, apoi URL-ul final.

In exercitiul 17 a trebuit sa folosesc response.history pentru ca ma uit la istoria redirectionarilor.

Aici e codul:

```
#Exercitiul 17

import requests
res_redir = requests.get(BASE_URL, timeout=TIMEOUT)
print("Istoric redirectionari:")
for r in res_redir.history:
   print(f"{r.status_code} → {r.url}")
print("URL final:", res_redir.url)
```

Aici e outputul:

```
Istoric redirectionari:
URL final: https://cybercor.org/
```

Deci nu a intrat nimic altundeva asa ca e doar un URL final.

Exercitiul 18. HEAD versus GET. Trimiteți o cerere către pagina principală cu `requests.head()` și una cu `requests.get()`. Comparați `len(response.content)` pentru ambele și explicați diferența într-un comentariu.

In exercitiul 18 a trebuit sad au request la head cu requests.head() si una simpla de requests.get() si sa le compar cu ajutorul la len(response.content).

Aici e codul:

```python
#Exercitiul 18

import requests
res_head = requests.head(BASE_URL, timeout=TIMEOUT)
res_get = requests.get(BASE_URL, timeout=TIMEOUT)
print("Dimensiunea corp la head:", len(res_head.content))
print("Dimensiunea corp la get:", len(res_get.content))

#head cere doar antenele paginii, fara sa descarce continutul
#get descarca tot continutul html al paginii
```

La fel are comentariu.

Aici e outputul:

```
Dimensiunea corp la head: 0
Dimensiunea corp la get: 1540
```

Exercitiul 19. Cookie-uri. Parcurgeți `response.cookies` și afișați, pentru fiecare cookie, atributele `.name` și `.secure`. Dacă nu există cookie-uri, afișați `Niciun cookie setat`.

In exercitiul 19 a trebuit sa gasesc cookies si sa le afisez.

Aici e codul in pyhton:

```python
#Exercitiul 19

import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
print("Cookiuri:")
if response.cookies:
    for c in response.cookies:
        print(f"Nume: {c.name}, Secure: {c.secure}")
else:
    print("Nici un cookie setat")
```

Si aici e outputul:

```
Cookiuri:
Nici un cookie setat
```

Exercitiul 20. Sesiuni. Creati un obiect `requests.Session()` și setați un `User-Agent` personalizat cu `session.headers.update({"User-Agent": "WebLab-<numele vostru>"})`. Trimiteți o cerere către `ECHO_URL + "/headers"` și confirmați în rezultat că antetul a fost trimis.

> **Verificati-vă:** atât `response.text`, cât și `response.json()` vă dau corpul răspunsului. De ce unul are paranteze, iar celălalt nu?

In exercitiul 20 a trebuit sad au request la sesiunea de pe BASE_URL personalizat cu session.headers.update({“User-Agent”: “WeLab-<numele>”}). Aceastav a trimite o cerere catre ECHO_URL + “/headers” si il voi printa.

Aici e codul:

```
#Exercitiul 20

import requests
session = requests.Session()
session.headers.update({"User-Agent": "WebLab-BodaciBogdan"})
res_sess = session.get(ECHO_URL + "/headers", timeout=TIMEOUT)
user_agent_primit = res_sess.json()["headers"].get("User-Agent")
print("User-Agent primit inapoi de la server:", user_agent_primit)
```

Aici e outputul la acest exercitiu:

```
User-Agent primit inapoi de la server: WebLab-BodaciBogdan
```

Partea 3 “functions”:

Exercitiul 21. Prima functie. Scrieți `fetch(url)`, care returnează obiectul răspuns. Apelați-o pentru `BASE_URL` și afișați codul de stare.

In exercitiul 21 a trebuit sa defiinesc o functie noua denumita fetch(url). In ea am dat return la requests.get(url, timeout=TIMEOUT) care dupa mai jos am scris ca response = fetch(BASE_URL) pentru a atribuii la url=BASE_URL si dupa i-am dat print.

Aici e codul facut:

```
#Exercitiul 21

import requests
def fetch(url):
        """Face o cerere get catre url si returneaza obiectul response"""
        return requests.get(url, timeout=TIMEOUT)
response = fetch(BASE_URL)
print("response status:", response.status_code)
```

Aici e outputul:

```
response status: 200
```

Exercitiul 22. Returnarea unei singure valori. Scrieți `get_status(url)`, care returnează doar codul de stare. Apelați-o pentru `/`, `/robots.txt` și `/sitemap.xml`, cu `time.sleep(1)` între apeluri.

In exercitiul 22 am create o functie noua numita get_status(url), in ea i-am atribuit la un response request.get(url,timeout) si am scris sa returneze status code. Mai jos am facut un un (cai) pentru a crea un path pentru ca sa pot apela la "/", "/robots.txt" si "/sitemap.xml" si cate un time.sleep(1) intre ele ca sa fie o pauza catre site. Am facut un loop for pentru ca sa gaseasca toate caile din get status la fiecare cale din cai si de fiecare data o sa dea time.sleep(1).

Aici e codul in python:

```python
#Exercitiul 22
print()
import requests
import time
def get_status(url):
            """Returneaza doar codul de stare http pentru adresa url """
            response = requests.get(url, timeout=TIMEOUT)
            return response.status_code
cai = ["/", "/robots.txt", "/sitemap.xml"]
for cale in cai:
        print(f"Status pentru {cale}: {get_status(BASE_URL + cale)}")
        time.sleep(1)
```

Aici e outputul:

```
Status pentru /: 200
Status pentru /robots.txt: 200
Status pentru /sitemap.xml: 200
```

Exercitiul 23. Parametri impliciti. Modificati functia în `fetch(url, timeout=10)`. Apelati-o o dată cu valoarea implicită și o dată cu `timeout=3`.

In exercitiul 23 a trebuit sa fac o functie fetch(url, timeout=10). Din vina ca am facut un fetch deja mai sus, l-am numit fetch_p3(url, timeout=10) si se termina ca sa returneze request.get(url,timeout=TIMEOUT). Mai jos am facut un response_implicit care ia valoare de fetch_p3(BASE_URL) si dupa mai jos response custom, res_custom pentru un timeout=3 si dupa le-am dat prin la ambele.

Aici e codul in python:

```python
#Exercitiul 23
print()
import requests
def fetch_p3(url, timeout=10):
            """Face o cerere get cu valoare implicita pentru timeout """
            return requests.get(url, timeout=TIMEOUT)
res_implicit = fetch_p3(BASE_URL)
res_custom = fetch_p3(BASE_URL, timeout=3)
print(f"Apelarea: {res_implicit}, {res_custom}")
```

Aici e outputul:

```
Apelarea: <Response [200]>, <Response [200]>
```

Exercitiul 24. O singură sarcină pe funcție. Scrieti `get_title(html)`, care primeste un șir HTML și returnează titlul paginii, refolosind logica din exercitiul 14. Observati că această funcție nu face nicio operatie de retea: descărcarea și analiza sunt acum separate.

Exercitiul 25. Docstring-uri. Adăugati un docstring funcției `get_title` și afișati-l cu `help(get_title)`.

Exercitiul 24 si 25 le-am facut impreuna din vina ca la 25 pur si simplu trebuia sa folosesc functia get_title din 24. La inceput am definit functia get_title(html) si dupa cum mi s-a fost spus m-am dus la exercitiul 14 din care am extras titlul “CYBERCOR” si l-am folosit aici. N-am inteles cum sa folosesc dar am facut sa nu returneze nimic.

Aici e codul:

```python
#Exercitiul 24 si 25
print()
import requests
def get_title(html):
            """Extrage si curata textul titlului dintrun cod html.
    parametri:
        html : codul html al paginii.
    returneaza:
        str: titlul curatat de spatii libere.
    """
            start = html.find("<title>")
            end = html.find("/<title>")
            if start ≠ -1 and end ≠ -1:
                    return html[start + len("<title>"):end].strip()
            return ""
```

Exercitiul 26. Adnotări de tip. Adăugati adnotări de tip (type hints) functiilor, de exemplu `def get_status(url: str) -> int:`. Apelati `get_status(123)` si observati ce se întâmplă. Vă opresc adnotările de tip?

In exercitiul 26 a trebuit sa defines o functie get_status(url:str) -> int:. Eu l-am numit get_status_typed, am facut doar programul ca definit si nu am mai facut nimic in el.

Iata codul:

```python
#Exercitiul 26
print()
import requests
def get_status_typed(url: str) → int:
            """Returneaza codul de stare folosind adnotari de tip"""
            response = requests.get(url, timeout=TIMEOUT)
            return response.status_code
```

Nu are output deoarece e doar def.

Exercitiul 27. Fără opriri neasteptate. Scrieti `page_exists(url)`, care returnează `True` sau `False`. Folositi `try` / `except requests.RequestException`, astfel încât functia să returneze `False` în loc să se oprească cu eroare, chiar si pentru un URL precum `https://this-domain-does-not-exist.invalid`.

In exercitiul 27 am definit functia page_exists(url) si am folosit un try si except in intreriorul sau pentru a vedea daca url care mi s-a fost transmis exista.

Aici e codul:

```python
#Exercitiul 27
print()
import requests

def page_exists(url):
    """Verifica daca o pagina exista si returneaza True sau False fara sa opreasca p>
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return res.status_code == 200
    except requests.RequestException:
        return False

print("Exista domeniul fals?:", page_exists("https://this-domain-does-not-exist.inva>
```

Aici e outputul:

```
Exista domeniul fals?: False
```

Exercitiul 28. Returnarea unui dictionar. Scrieți `check_paths(base, paths)`, care primește o listă precum `["/", "/robots.txt", "/sitemap.xml"]` și returnează un dicționar `{path: status_code}`.

In exercitiul 28 am definit functia check_paths(base, paths) pentru ca sa primeasca liste de la [“/”, “/robots.txt”, “/sitemap.xml”] si sa returneze dictionarul path cu status code.

Aici e codul in python:

```python
#Exercitiul 28
print()
import requests
import time
def check_paths(base,paths):
        """Primeste o lista de cai si returneaza un dictionar cu statusul fiecareia>
        rezultate = {}
        for p in paths:
              url_complet = base.rstrip("/") + p
              try:
                 response = requests.get(url_complet, timeout=TIMEOUT)
                 rezultate[p] = response.status_code
              except requests.RequestsException:
                  rezultate[p] = 0
              time.sleep(1)
        return rezultate

print("Verificare cai:", check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"]))
```

Aici e outputul:

```
Verificare cai: {'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}
```

Exercitiul 29. Argumente cu nume. Scrieti `get_header(url, name, default="lipseste")`, care returnează valoarea unui singur antet. Apelati-o cu argumente cu nume: `get_header(BASE_URL, name="Server")`.

In exercitiul 29 a trebuit sa defines functia get_header(url, name, default=”lipseste”) care sa returneze un singur antet apleand doar la get_header. Am facut cu try si except inauntru la def. Deci am facut un request.get legal la response ca sa ii ia valoarea si sa dea return la res.headers.get(name, defaults) pentru ca sa gaseasca o valoare specifica. Dupa la except sa returneze defaults is dupa i-am dat print.

Aici e programul in python:

```python
#Exercitiul 29
print()
import requests
def get_header(url, name, defaults="lipseste"):
            """Returneaza valoarea unui singur antet din raspuns."""
            try:
                response = requests.get(url, timeout=TINMEOUT)
                return res.headers.get(name, defaults)
            except:
                return defaults
print("antet server cu argument numit:", get_header(BASE_URL, name="Server"))
```

Aici e outputul la program:

```
antet server cu argument numit: lipseste
```

Exercitiul 30. Antete de securitate. Scrieti `security_headers(url)`, care verifică prezenta acestor cinci antete si returnează `{antet: True/False}`:

- `Strict-Transport-Security`
- `Content-Security-Policy`
- `X-Frame-Options`
- `X-Content-Type-Options`
- `Referrer-Policy`

In exercitiul 30 a trebuit sa defines o functie security_headers(url) care sa verifice daca fiecare antet si sa returneze cate din ele merg. Deci am bagat antetele intr-o singura variabila antete_verificate si dupa am am dat try care daca merge sa adauge succesul in lista, in except sa fie false si sa nu se adauge in lista aia care a dat success.

Aici e codul si outputul urmeaza sa fie in urmatorul exercitiu:

```python
#Exercitiul 30
print()
import requests
def security_headers(url):
    """Verifica prezenta celor 5 antete de securitate."""
    antete_verificate = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy"
    ]
    try:
        res = requests.get(url, timeout=TIMEOUT)
        return {h: h in res.headers for h in antete_verificate}
    except requests.RequestException:
        return {h: False for h in antete_verificate}
```

Exercitiul 31. Funcții care apelează funcții. Scrieți `score_headers(results)`, care primește dicționarul din exercițiul 30 și returnează un șir precum `"3/5"`. Apelați-le împreună: `score_headers(security_headers(BASE_URL))`.

In exercitiul 31 a trebuit sa folosesc security headers din exercitiul 30, In acelasi timp aici se vor aduna la lista de success fiecare am create un gasite care sa sumeze toate rezultatele successful. Dupa sa returneze tot ce a fost gasit si lungimea la result. Dupa am dat print la ea si am folosit score_headers(security_headers(BASE_URL)) pentru ca sa se duca la cybercor.

Aici e codul care functioneaza:

```python
#Exercitiul 31
print()
import requests
def score_headers(results):
        """Calculeaza scorul antetelor de securitate in format '3/5'."""
        gasite = sum(1 for prezent in results.values() if prezent)
        return f"{gasite}/{len(results)}"
print("Scor securitate cybercor:", score_headers(security_headers(BASE_URL)))
```

Si aici e outputul:

```
Scor securitate cybercor: 3/5
```

Exercitiul 32. Două funcții, o sarcină. Scrieți `fetch_robots(base)`, care returnează textul fișierului `/robots.txt` sau `None` dacă fișierul lipsește. Apoi scrieți `disallowed_paths(robots_text)`, care returnează o listă cu toate valorile `Disallow:`. Asigurați-vă că funcționează și atunci când primește `None`.

In exercitiul 32 a trebuit sa defines functia fetch_robots(base) care sa returneze textul fisierului /robots.txt sau None. Apoi sa creez o functie noua definite si sa o numesc dissalowed_paths(robots_text) si sa fac o lista cu toate valorile dissalow.

Aici e codul in python:

```python
# Exercitiul 32
print()
import requests
def fetch_robots(base):
    """Descarca robots.txt sau returneaza None daca nu il gaseste."""
    url = base.rstrip("/") + "/robots.txt"
    try:
        res = requests.get(url, timeout=TIMEOUT)
        if res.status_code == 200:
            return res.text
        return None
    except requests.RequestException:
        return None
def disallowed_paths(robots_text):
    """Extrage o lista cu toate caile Disallow: din robots.txt."""
    if not robots_text:
        return []
    cai_interzise = []
    for linie in robots_text.splitlines():
        linie = linie.strip()
        if linie.lower().startswith("disallow:"):
            parti = linie.split(":", 1)
            if len(parti) > 1 and parti[1].strip():
                cai_interzise.append(parti[1].strip())
    return cai_interzise
print("Cai interzise in robots.txt:", disallowed_paths(fetch_robots(BASE_URL)))
```

Aici e outputul printat:

```
Cai interzise in robots.txt: ['/', '/', '/', '/', '/', '/', '/', '/', '/', '/', '/',
'/', '/', '/', '/', '/', '/', '/', '/', '/', '/', '/', '/', '/', '/', '/', '/',
'/', '/', '/']
```

Exercitiul 33. Oricâte argumente. Scrieți `response_times(*urls)`, care returnează un dictionar `{url: secunde}`. Apelati-o cu două URL-uri.

In exercitiul 33 a trebuit sa defines o functie response_times(*urls) care sa returneze un dicitonar url cu cate secunde a fost apelat pana a ajuns cererea.

Aici e codul:

```python
# Exercitiul 33
print()
import time
import requests
def response_times(*urls):
    """Masoara timpul de raspuns pentru oricate URL-uri primeste."""
    rezultate = {}
    for url in urls:
        t_start = time.perf_counter()
        try:
            requests.get(url, timeout=TIMEOUT)
            t_total = time.perf_counter() - t_start
            rezultate[url] = round(t_total, 4)
        except requests.RequestException:
            rezultate[url] = -1.0
        time.sleep(1)
    return rezultate
print("Timpi de raspuns:", response_times(BASE_URL, ECHO_URL))
```

Aici e outputul printat:

```
Timpi de raspuns: {'https://cybercor.org': 0.1207, 'https://httpbin.org': 0.4585}
```

Exercitiul 34. Argumente cu nume colectate. Scrieți `log(message, **details)`, care afișează mesajul urmat de fiecare detaliu. Exemplu: `log("verificat", url=BASE_URL, status=200)` afișează `verificat | url=https://cybercor.org | status=200`.

> **Verificați-vă:** care este diferența dintre a afișa o valoare cu `print()` în interiorul unei funcții și a o returna cu `return`?

In exercitiul 34 am create o functie definite log(message, **details) care sa afiseze mesajul la fiecare detaliu. Si sa verifice url status la cybercor.

Aici e codul in python:

```python
# Exercitiul 34
print()
def log(message, **details):
    """Afiseaza mesajul si detaliile primite ca argumente cu nume."""
    detalii_formatate = " | ".join(f"{k}={v}" for k, v in details.items())
    if detalii_formatate:
        print(f"{message} | {detalii_formatate}")
    else:
        print(message)
log("verificat", url=BASE_URL, status=200)
```

Aici e outputul:

```
verificat | url=https://cybercor.org | status=200
```

Partea 4 “modules”:

Exercitiul 35. Descompunerea unui URL. Folosiți `urllib.parse.urlparse()` pe `https://cybercor.org/path?x=1#top`. Afișați `.scheme`, `.netloc`, `.path`, `.query` și `.fragment`.

In exercitiul 35 a trebuit sa folosesc urllib.parse.urlparse pe linku la cybercor si trebuie sa afisesz .scheme, .netloc, .path, .query si .fragment.

Aici e codul in python:

```python
#Exercitiul 35

import urllib.parse
parsed = urllib.parse.urlparse("https://cybercor.org/path?x=1#top")
print("scheme:", parsed.scheme)
print("netloc:", parsed.netloc)
print("path:", parsed.path)
print("query:", parsed.query)
print("fragment:", parsed.fragment)
```

Aici e outputul:

```
scheme: https
netloc: cybercor.org
path: /path
query: x=1
fragment: top
```

Exercitiul 36. Compunerea URL-urilor. Folosiți `urllib.parse.urljoin()` pentru a transforma legăturile relative `/about`, `contact.html` și `../index.html` în URL-uri complete, pornind de la `BASE_URL`.

In exercitiul 36 trebuie sa folosesc urllib.parse.urljoin() pentru a uni BASE_URL cu /about, contact.html si ../index.html.

Aici e codul in python:

```python
#Exercitiul 36
print()
import urllib.parse
for legatura in ["/about", "contact.html", "../index.html"]:
    url_absolut = urllib.parse.urljoin(BASE_URL, legatura)
    print(f"Legatura transformata pentru {legatura}: {url_absolut}")
```

Aici e outputul:

```
Legatura transformata pentru /about: https://cybercor.org/about
Legatura transformata pentru contact.html: https://cybercor.org/contact.html
Legatura transformata pentru ../index.html: https://cybercor.org/index.html
```

Exercitiul 37. Extragerea legăturilor. Folositi `re.findall(r'href="([^"]+)"', html)` pentru a obtine toate legăturile de pe pagina principală. Puneti codul într-o functie `extract_links(html)`, care returnează legăturile fără duplicate.

In exercitiul 37 folosesc modulul re, si folosesc re.findall pentru a obtine toate legaturile de la pagina principala, in acelasi timp trebuie sa creez o functie definite extract_links care sa returneze legaturile fara duplicate.

Aici e codul in python:

```python
#Exercitiul 37
print()
import re
def extract_links(html):
    """Extrage toate legaturile href din html"""
    legaturi = re.findall(r'href="([^"]+)"', html)
    return list(set(legaturi))
```

Nu am facut outputul.

Exercitiul 38. Interne sau externe? Scrieti `split_links(links, domain)`, care returnează două liste: legăturile de pe acelasi domeniu și legăturile către alte site-uri. Folositi mai întâi `urljoin()`, apoi `urlparse().netloc`.

In exercitiul 38 a trebuit sa fac o functie definite split_links(links, domain) care returneaza 2 liste. Si trebuie sa folosesc mai intai urljoin() si apoi urlparse().netloc care mai intai transforma url intr-un format de cod cu care poti lucre cu ajutorul la parse si dupa cu netloc extrage locatia la network de pe url.

Aici e codul python care merge:

```python
#Exercitiul 38
print()
import urllib.parse
def split_links(links,domain):
    """Separa legarurile interne si externe"""
    interne = []
    externe = []
    domeniu_baza = urllib.parse.urlparse(domain).netloc

    for link in links:
        url_complet = urllib.parse.urljoin(domain, link)
        domeniu_link = urllib.parse.urlparse(url_complet).netloc
        if domeniu_link == domeniu_baza:
            interne.append(url_complet)
        else:
            externe.append(url_complet)
    return interne, externe
```

Nu i-am facut output.

Exercitiul 39. Propriile metode. Creati o clasă care moșteneste `html.parser.HTMLParser` și suprascrieti metoda `handle_starttag()`, pentru a colecta atributul `src` al fiecărui tag `<img>`.

```python
from html.parser import HTMLParser

class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        # codul vostru aici
        pass
```

> **Indiciu:** `attrs` este o listă de perechi `(nume, valoare)`, de exemplu `[("src", "/logo.png"), ("alt", "Logo")]`. Expresia `dict(attrs).get("src")` vă dă valoarea atributului `src` sau `None`.

In exercitiul 39 a trebuit sa fac o clasa care sa suprascrie handle_starttag() pentru ca sa colecteze atributul src de la fiecare tag <img>.

Aici e prima parte de cod:

```python
#Exercitiul 39
print()
from html.parser import HTMLParser
import requests
class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []
    def handle_starttag(self, tag, attrs):
        if tag.lower() == "img":
            valoare_src = dict(attrs).get("src")
            if valoare_src:
                self.images.append(valoare_src)
```

Aici e partea a doua cu TODO:

Cum testati exercitiul 39. Testați mai întâi parserul pe un fragment HTML scris de voi, pentru care știți dinainte rezultatul corect. Abia după aceea rulați-l pe site-ul real. Adăugați acest test sub clasa voastră:

```python
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",                        # imagine obisnuită
    "poza.jpg",                         # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile asteptate"
print("Testul a trecut!")
```

Dacă testul trece, parserul vostru: găsește tag-urile `<img>` indiferent de majuscule, ignoră tag-urile `<img>` fără `src`, ignoră celelalte tag-uri (de exemplu `<a>`) și păstrează ordinea din pagină. Dacă `assert` eșuează, comparați lista afișată de `print()` cu lista așteptată.

Si aici e codul acesta prin urmare:

```
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",  # imagine obișnuită
    "poza.jpg",  # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")
```

Si aici e ultima parte:

Apoi rulați parserul pe pagina reală:

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
```

Verificați rezultatul în două moduri: comparați numărul găsit cu `response.text.lower().count("<img")` și deschideți pagina în browser, apoi apăsați Ctrl+U ca să vedeți codul sursă. O listă goală nu înseamnă neapărat o greșeală: unele site-uri afișează imaginile prin CSS sau SVG, nu prin tag-uri `<img>`. Tocmai de aceea testul cu HTML cunoscut vine primul.

Exercitiul 40. Amprenta paginii. Scrieți `page_fingerprint(url)`, care returnează `hashlib.sha256(response.content).hexdigest()`. Rulați-o de două ori și comparați rezultatele. Când ar fi diferite?

Si aici e ultima parte din cod:

```
res_real = requests.get(BASE_URL, timeout=TIMEOUT)
finder_real = ImageFinder()
finder_real.feed(res_real.text)
print(len(finder_real.images), "imagini gasite pe site-ul real")
```

Aici e outputul:

```
['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!
0 imagini gasite pe site-ul real
```

Exercitiul 40. Amprenta paginii. Scrieți `page_fingerprint(url)`, care returnează `hashlib.sha256(response.content).hexdigest()`. Rulați-o de două ori și comparați rezultatele. Când ar fi diferite?

In exercitiul 40 am definit functia page_fingerprint(url) care sa returneze hashlib. Si care trebuia sa o rulez de 2 ori. Am facut 2 hashuri care da run 2 ori si le compar.

Aici e codul in python:

```python
#Exercitiu 40
print()
import hashlib
import requests
def page_fingerprint(url):
    """Returneaza hash SHA-256 al continutului paginii"""
    res = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(res.content).hexdigest()
print("Hash 1:", page_fingerprint(BASE_URL))
print("Hash 2:", page_fingerprint(BASE_URL))

#Rezultatele pot fi diferite daca intre timp sa schimbat ceva pe server
```

Aici e outputul:

```
Hash 1: 7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
Hash 2: 7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
```

Exercitiul 41. Salvarea în JSON. Salvați antetele paginii principale în `headers.json` cu `json.dump(dict(response.headers), file, indent=2)`. Apoi încărcați fișierul înapoi cu `json.load()` și afișați un antet din el.

In exercitiul 41 trebuie sa folosesc json dump pentru a da outputul intrun file json care dupa trebuia sad au json.load pentru ca sa il afisez.

Aici e codul in python:

```python
#Exeritiul 41
print()
import json
import requests
res_headers = requests.get(BASE_URL, timeout=TIMEOUT)
with open("header.json","w", encoding="utf-8") as f:
    json.dump(dict(res_headers.headers), f, indent=2)
with open("header.json","r", encoding="utf-8") as f:
    date_incarcate = json.load(f)
print("Antet incarcat din fisier:", date_incarcate.get("Content-Type"))
```

Aici e outputul:

```
Antet incarcat din fisier: text/html; charset=utf-8
```

Exercitiul 42. Interogare DNS. Scrieti `resolve(hostname)`, care returnează `socket.gethostbyname(hostname)`. Afisați adresa IP a domeniului `cybercor.org`.

In exercitiul 42 am definit functia resolve(hostname) care sa returneze socket.gethostbyname(hostname) si dupa sa afisez adresa IP de pe cybercor.

Aici e codul care merge:

```
#Exercitiul 42
print()
import socket
def reslove(hostname):
    """Afla adresa IP pentru un nume de domeniu"""
    return socket.gethostbyname(hostname)
print("IP cybercor.org:", reslove("cybercor.org"))
```

Aici e outputul:

```
IP cybercor.org: 172.67.211.96
```

Exercitiul 43. Expirarea certificatului. Scrieti `cert_days_left(hostname)`, care se conectează la portul 443, citeste certificatul și returnează câte zile mai sunt până la expirarea lui.

> **Indiciu:** folosiți `ssl.create_default_context()`, `socket.create_connection((hostname, 443))`, `context.wrap_socket(sock, server_hostname=hostname)` și `getpeercert()["notAfter"]`. Convertiti data cu `ssl.cert_time_to_seconds()`.

> **Verificați-vă:** în exercitiul 39, cine apelează `handle_starttag()`? Voi nu o apelați niciodată direct.

In exercitiul 43 a trebuit sa defines functia cet_days_left(hostname) care sa se conecteze la portul 443, care sa citeasca certificatul si sa spuna cate zile a ramas pana expira.

Aici e codul:

```
#Exercitiul 43
print()
import socket
import ssl
import datetime
def cert_days_left(hostname):
    """Citeste certificatul ssl si returneaza cate zile mai sunt valabile"""
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443)) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert = ssock.getpeercert()
            timestamp = ssl.cert_time_to_seconds(cert["notAfter"])
            data_expirare = datetime.datetime.fromtimestamp(timestamp, tz=datetime.t>
            acum = datetime.datetime.now(tz=datetime.timezone.utc)
            return (data_expirare - acum).days
print("zile ramase certificat:", cert_days_left("cybercor.org"))
```

Aici e outputul:

```
zile ramase certificat: 80
```

Exercitiul 44. Crearea unui modul. Mutati `fetch`, `get_status`, `get_title` și `security_headers` într-un fisier nou, `webtools.py`. În `main.py`, scrieți `import webtools` și apelati `webtools.get_title(webtools.fetch(BASE_URL).text)`.

In exercitiul 44 trebuia sa creez un modul care sa mute fetch, get_status, get_title si security_headers intr-un fisier webtools.py. In main.py sa scriu import webtools si sa apelez la acea functie.

Aici e codul la execritiul in webtools.py:

```python
#Exercitiul 44
print()
def get_title(html):
    """Extrage titlul dintrun text html"""
    start = html.find("<title>")
    end = html.find("</title>")
    if start ≠ -1 and end ≠ -1:
        return html[start + len("<title>"):end].strip()
    return "Fara titlu"
```

Exercitiul 45. Importarea anumitor nume. În `main.py`, treceți la `from webtools import get_title, security_headers`. Într-un comentariu, explicati ce s-ar întâmpla dacă `main.py` ar avea și o functie proprie numită `get_title`.

Aici e in main.py :

```python
#Exercitiul 44 si 45

from webtools import get_title, security_headers

#Daca main.py ar fi defini o functie locala cu acelasi nume,
#functia importanta din webtools ar fi suprascrisa.
```

Exercitiul 46. Blocul main. Adăugati acest bloc la finalul fisierului `webtools.py`:

```
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Rulati `python webtools.py`, apoi `python main.py`. Explicati de ce autotestul rulează doar în primul caz.

Acum inapoi in in webtools:

```
#Exercitiul 46, blocul main pentru autotest
print()
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

```
#Exercitiul 44 si 46
print()
def get_status(url):
    """Returneaza codul de stare http"""
    import requests
    try:
        res = fetch(url)
        return res.status_code
    except:
        return 0
```

Exercitiul 47. Constante de modul. Adăugati constanta `DEFAULT_HEADERS = {"User-Agent": "WebLab-<numele vostru>"}` în `webtools.py` si folositi-o în `fetch()`. În `main.py`, importati constanta si afisati-o.

Aici e codul in main.py:

```
#Exercitiul 47

from webtools import DEFAULT_HEADERS
print("DEAFULT_HEADERS importat:", DEFAULT_HEADERS)
```

Aici e in webtools.py:

```
#Exercitiul 44 si 47

def fetch(url, timeout=TIMEOUT):
    """Face o cerere get cu antetele noastre implicate"""
    import requests
    return requests.get(url, headers=DEFAULT_HEADERS, timeout=timeout)
```

Exercitiul 48. Argumente din linia de comandă. Folositi `argparse` în `main.py`, astfel încât programul să ruleze ca `python main.py https://cybercor.org`. Transmiteti argumentul URL functiilor voastre, în loc să folositi `BASE_URL`.

Aici e in main.py:

```python
#Exercitiul 48

import argparse
parser = argparse.ArgumentParser(description="Analizor site web")
parser.add_argument("url", nargs="?", default="https://cybercor.org", help="url de a>
args = parser.parse_args()
target_url = args.url
```

Exercitiul 49. Un raport CSV. Folositi modulul `csv` pentru a salva rezultatul functiei `check_paths()` în `report.csv`, cu coloanele `path`, `status` și `checked_at`. Pentru oră, folositi `datetime.now().isoformat()`.

Aici e codul in main.py:

```python
#Exercitiul 49

import csv
import datetime
import webtools
cai = ["/", "/robots.txt", "/sitemap.xml"]
with open("report.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["path", "status", "checked_at"])
    for c in cai:
        st = webtools.get_status(target_url.rstrip("/") + c)
        writer.writerow([c, st, datetime.datetime.now().isoformat()])
print("Raportul csv a fost salvat in report.csv")
```

Exercitiul 50. Proiect final: raportul site-ului. Scrieți `site_report(url)` în `webtools.py`. Functia trebuie să folosească functiile voastre pentru a colecta toate informațiile de mai jos, să le afiseze și să le salveze în `report.json`:

1. Codul de stare și URL-ul final
2. Titlul paginii
3. Adresa IP
4. Lantul de redirectionări pornind de la `http://`
5. Scorul antetelor de securitate, de exemplu `3/5`
6. Zilele rămase până la expirarea certificatului
7. Numărul de legături interne și externe

8. Căile interzise din `robots.txt`

Aici e codul in main.py:

```
#Exercitiul 50

import webtools
webtools.site_report(target_url)
```

Aici e tot restul in webtools:

```
#exercitiul 44 si 50
print()
def security_headers(url):
    """Verifica antetele de securitate"""
    import requests
    antete = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy",
    ]
    try:
        res = fetch(url)
        return {h: h in res.headers for h in antete}
    except:
        return {h: False for h in antete}
```

```
#Exercitiul 50, scor antete
print()
def score_headers(results):
    """Calculeaza scorul de securitate"""
    return f"{sum(1 for v in results.values() if v)}/{len(results)}"
```

```
#Exercitiul 50, preluare robot.txt
print()
def fetch_robots(base):
    """Descarca fisierul /robots.txt"""
    import requests
    try:
        res = fetch(base.rstrip("/") + "/robots.txt")
        return res.text if res.status_code == 200 else None
    except requests.RequestException:
        return None
```

```
#Exercitiul 50, cai interzise
print()
def disallowed_paths(robots_text):
    """Extrage caile disallow din robots.txt"""
    if not robots_text:
        return []
    paths = []
    for line in robots_text.splitlines():
        if line.strip().lower().startswith("disallow"):
            parts = line.split(":", 1)
            if len(parts) > 1 and parts[1].strip():
                paths.append(parts[1].strip())
    return paths
```

```python
#Exercitiul 50, zile certificate ssl
print()
def cert_days_left(hostname):
    """Returneaza zilele ramase pentru certificatul ssl"""
    import socket
    import ssl
    import datetime
    context = ssl.create_default_context()
    try:
        with socket.create_connection((hostname, 443)) as sock:
            with context.wrap_socket(sock, server_hostname=hostname) as ssock:
                cert = ssock.getpeercert()
                exp = datetime.datetime.fromtimestamp(ssl.cert_time_to_seconds(cert[
                return (exp - datetime.datetime.now(tz=datetime.timezone.utc)).days
    except Exception:
        return 0
```

```python
#Exercitiul 50 final
print()
def site_report(url):
    """Genereaza raportul complet al sitului si il salveaza in report.json"""
    import urllib.parse
    import socket
    import re
    import json
    parsed = urllib.parse.urlparse(url)
    hostname = parsed.netloc or parsed.path
    res = fetch(url)
    status_code = res.status_code
    final_url = res.url
    titlu = get_title(res.text)
    try:
        ip = socket.gethostbyname(hostname)
    except:
        ip = "N/A"

    try:
        res_http = fetch(f"http://{hostname}")
        redirects = [f"{r.status_code} → {r.url}" for r in res_http.history]
        redir_text = ", ".join(redirects) if redirects else f"Niciuna ({res_http.sta
    except Exception:
        redir_text = "Eroare rediriconare"
    sec_res = security_headers(url)
    sec_score = score_headers(sec_res)
    zile_cert = cert_days_left(hostname)
    raw_links = list(set(re.findall(r'href="([^"]+)"', res.text)))
    interne, externe = [], []
    for l in raw_links:
```

```python
        full = urllib.parse.urljoin(BASE_URL, l)
        if urllib.parse.urlparse(full).netloc == hostname:
            interne.append(full)
        else:
            externe.append(full)

    robots = fetch_robots(url)
    interzise = disallowed_paths(robots)
    raport = {
        "target_url": url,
        "status_code": status_code,
        "final_url": final_url,
        "title": titlu,
        "ip_address": ip,
        "redirects": redir_text,
        "security_score": sec_score,
        "cert_days_left": zile_cert,
        "internal_links_count": len(interne),
        "external_links_count": len(externe),
        "disallowed_paths": interzise
    }
    with open("report.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print(f"Raport site: {url}")
    print(f"Cod de stare: {status_code}")
    print(f"Titulu: {titlu}")
    print(f"Adresa IP: {ip}")
    print(f"Redirectionare: {redir_text}")
    print(f"Scor securitate: {sec_score}")
    print(f"Certificat: {zile_cert} de zile ramase")
    print(f"Legaturi: {len(interne)} interne, {len(externe)} externe")
    print(f"Cai interzise: {', '.join(interzise) if interzise else 'Niciuna'}")
    print("Salvat in report.json")
    return raport
```

Aici e outputul la main.py:


DEAFULT_HEADERS importat: {'User-Agent': 'WebLab-BodaciBogdan'}
Raportul csv a fost salvat in report.csv
Raport site: https://cybercor.org
Cod de stare: 200
Titulu: CYBERCOR
Adresa IP: 172.67.211.96
Redirectionare: 301 → http://cybercor.org/
Scor securitate: 3/5
Certificat: 80 de zile ramase
Legaturi: 1 interne, 0 externe
Cai interzise: /, /, /,

[1] Python Software Foundation. Python 3.12 Documentation — Standard Library Reference. https://docs.python.org/3/

[2] Requests Developers. Requests: HTTP for Humans (v2.31.0 documentation). https://requests.readthedocs.io/

[3] Kenneth Reitz. httpbin.org: HTTP Request & Response Service. https://httpbin.org/

[4] Universitatea Tehnică a Moldovei. Ghid de laborator: Funcții, metode și importuri pe web. Chișinău, 2026.